

International University

School of Computer Science & Engineering

Net-Centric Programming

IT096IU

Final Report

Clash Royale

Submitted by

Lê Quan Phát Thành – ITITI21312

Lê Phương Nghi – ITITI21253

Group: 17

Date Performed: 18/04/2025

Date Submitted: 01/06/2025

Course Instructor: L.T.Son

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
LIST OF FIGURES	3
CHAPTER I.....	5
I. Overview	5
1. About us.....	5
2. Purpose & goals.....	5
3. Technology Stack	6
CHAPTER II.....	7
I. Client-Server Architecture	7
II. Component Overview.....	7
1. Server Component	7
a. TCP Server Structure.....	7
• Manages TCP connections and client handling	7
• Maintains a map of connected clients	7
• Handles game session management	7
• Implements thread-safe operations using mutex	7
2. Game Manager	10
III. Game State Management.....	12
CHAPTER 3	14
I. Game Modes	14

II.	Game Elements.....	15
1.	Towers	15
2.	Troops	15
III.	Combat System.....	15
1.	Turn Sequence	15
2.	Battlefield Visualization	16
CHAPTER IV		17
I.	Discussion	17
II.	Future Improvements	18
III.	References	19

LIST OF FIGURES

Figure 1 – TCP Server Structure.....	8
Figure 2 – Client Structure	8
Figure 3 – Server Initialization	9
Figure 4 – Command Handling.....	9
Figure 5 – Manager Structure	10
Figure 6 – Game Creation.....	11
Figure 7 – Player Management	12
Figure 8 – Game End Handling	13
Figure 9 – Time Management.....	13

Figure 10 - Simple mode.....14

Figure 11 – Enhanced mode.....14

Figure 12 – Tower elements.....15

Figure 13 – Troop elements15

Figure 14 – Turn Sequence15

Figure 15 – Battlefield Visualization16

CHAPTER I

INTRODUCTION

I. Overview

1. About us

Text-Based Clash Royale (TCR) is a multiplayer strategy game inspired by the popular Clash Royale title, but implemented entirely in a terminal-based environment using Golang. The game replicates the core strategic mechanics of Clash Royale—including tower defense, troop deployment, and damage calculation—while leveraging a simple text interface for both interaction and visualization. TCR aims to provide an engaging tactical experience while showcasing how a real-time or turn-based multiplayer game can be built with basic networking principles.

2. Purpose & goals

The primary purpose of this project is to explore how core gameplay features of a modern mobile strategy game can be translated into a simplified, text-based version. The goals include:

- Designing a functional multiplayer game with both turn-based and real-time gameplay modes.
- Implementing network communication over TCP using Go's standard libraries.
- Structuring the game with clear modular separation (server, client, models, game logic).
- Providing an extensible architecture for future enhancements like new troop types or advanced battle mechanics.

By focusing on these goals, TCR serves both as a fun game and as an educational demonstration of systems programming, game logic, and network communication.

3. Technology Stack

The project is developed in **Golang** due to its excellent support for concurrency and networking, making it well-suited for a multiplayer game. The communication between players and the server is built on **TCP sockets**, ensuring reliable transmission of gameplay messages in both turn-based and real-time modes.

Key technologies and tools include:

- **Golang**: for all backend logic and networking
- **TCP Networking**: for reliable client-server communication
- **Goroutines and Channels**: to handle concurrent matches and player interactions
- **Standard I/O (text-based interface)**: for interaction with the game without a graphical frontend

This introduction sets the stage for the architectural and gameplay details that follow in subsequent chapters.

CHAPTER II

SYSTEM ARCHITECTURE

I. Client-Server Architecture

TCR follows a classic **client-server model**, where multiple clients (players) connect to a centralized game server. The server is responsible for managing player connections, handling game sessions, synchronizing game states, and enforcing rules. This architecture ensures a consistent game state across clients and supports both turn-based and real-time gameplay.

- **Server:** Acts as the game coordinator. It handles incoming client requests, processes game logic, maintains global game states, and broadcasts updates.
- **Client:** Represents an individual player. It sends commands to the server (e.g., deploy troop, attack), receives state updates, and renders them in the terminal interface.

II. Component Overview

The system is divided into multiple modular components to improve maintainability and clarity:

1. Server Component

a. TCP Server Structure

- Manages TCP connections and client handling
- Maintains a map of connected clients
- Handles game session management
- Implements thread-safe operations using mutex

```
// TCPServer represents a TCP server for the game
type TCPServer struct {
    listener      net.Listener
    manager       *game.Manager
    clients       map[string]*Client
    mutex         sync.Mutex
    gameIDGen     int
    waitingForGame *Client
}
```

Figure 1 – TCP Server Structure

b. Client Structure

- Manages individual client connections
- Handles client authentication (login/register)
- Processes client commands
- Manages message sending with thread safety

```
// Client represents a connected client
You, 3 weeks ago | 1 author (You)
type Client struct {
    conn      net.Conn
    username  string
    player    *models.Player
    server    *TCPServer
    sendMutex sync.Mutex
}
```

Figure 2 – Client Structure

c. Server Initialization

```

// NewTCPServer creates a new TCP server
func NewTCPServer(address string) (*TCPServer, error) {
    listener, err := net.Listen("tcp", address)
    if err != nil {
        return nil, err
    }

    return &TCPServer{
        listener: listener,
        manager:  game.NewManager(),
        clients:  make(map[string]*Client),
        mutex:    sync.Mutex{},
        gameIDGen: 1,
    }, nil
}

```

Figure 3 – Server Initialization

d. Command Handling

```

128 // handleCommand handles a command from the client
129 func (c *Client) handleCommand(line string) error {
130     defer func() {
131         if r := recover(); r != nil {
132             log.Printf("Recovered from panic in handleCommand: %v", r)
133         }
134     }()
135
136     parts := strings.Split(line, " ")
137     if len(parts) == 0 {
138         return fmt.Errorf("empty command")
139     }
140
141     command := strings.ToUpper(parts[0])
142     args := parts[1:]
143
144     switch command {
145     case "LOGIN":
146         return c.handleLogin(args)
147     case "REGISTER":
148         return c.handleRegister(args)
149     case "PLAY":
150         return c.handlePlay(args)
151     case "DEPLOY":
152         return c.handleDeploy(args)
153     case "HELP":
154         return c.handleHelp()
155     case "STATS":
156         return c.handleStats()
157     case "TROOPS":
158         return c.handleTroops()
159     case "LEVEL":
160         return c.handleLevel()
161     default:
162         return fmt.Errorf("unknown command: %s", command)
163     }
164 }
165

```

Figure 4 – Command Handling

2. Game Manager

a. Core Game Logic

- Manages game states and player sessions
- Handles turn-based gameplay
- Implements both simple and enhanced game modes
- Manages player statistics and experience system

```
// Manager is responsible for managing the game state  
You, 3 weeks ago | 1 author (You)  
type Manager struct {  
    Games          map[string]*models.GameState  
    Players        map[string]*models.Player  
    GamesMutex     sync.Mutex  
    ClientConnections map[string]Client  
}
```

Figure 5 – Manager Structure

```
// CreateGame creates a new game with the given mode and players
func (m *Manager) CreateGame(gameID string, player1, player2 string, gameMode string) (*models.GameState, error) {
    m.GamesMutex.Lock()
    defer m.GamesMutex.Unlock()

    if _, exists := m.Games[gameID]; exists {
        return nil, fmt.Errorf("game with ID %s already exists", gameID)
    }

    p1, ok1 := m.Players[player1]
    p2, ok2 := m.Players[player2]

    if !ok1 || !ok2 {
        return nil, fmt.Errorf("one or both players not found")
    }

    // Log player levels before reset
    log.Printf("Creating game with player1: %s (Level %d) and player2: %s (Level %d)",
        p1.Username, p1.Level, p2.Username, p2.Level)

    // Reset player state for a new game
    m.resetPlayerForGame(p1)
    m.resetPlayerForGame(p2)

    // Create the game state
    game := &models.GameState{
        Players:    [2]*models.Player{p1, p2},
        CurrentTurn: 0, // Player 1 starts
        TimeLeft:   180, // 3 minutes
        GameMode:   gameMode,
    }

    m.Games[gameID] = game
}
```

```
// Notify players
m.notifyPlayers(gameID, fmt.Sprintf("Game created: %s vs %s (Mode: %s)", player1, player2, gameMode))
m.notifyPlayers(gameID, fmt.Sprintf("Game started! %s's turn", player1))

// Show instructions for simple mode
if gameMode == "simple" {
    m.notifyPlayers(gameID, "\n===== BATTLE GUIDE =====")
    m.notifyPlayers(gameID, "\n• Players start with 3 units & 3 defensive structures (1 Main, 2 Secondary)")
    m.notifyPlayers(gameID, "\n• Secondary Defense 1 must fall before targeting Secondary Defense 2 or Main Defense")
    m.notifyPlayers(gameID, "\n• Units that destroy a structure may immediately launch another attack")
    m.notifyPlayers(gameID, "\n• Command: DEPLOY <troops_number> <tower_number> to begin assault")
    m.notifyPlayers(gameID, "\n• Command: STATS to view current units and defenses")
    m.notifyPlayers(gameID, "\n")

    // Highlight that it's player 1's turn
    m.notifyPlayers(gameID, "\n-----")
    m.notifyPlayers(gameID, fmt.Sprintf("It's now %s's turn", player1))
    m.notifyPlayers(gameID, "\n")
}

// Display the initial stats to show level bonuses are applied
m.displayContinuousStats(gameID)

battlefield := m.NewBattlefieldMap(gameID)
battlefield.drawMap()

return game, nil
}
```

Figure 6 – Game Creation

b. Battle System

- Controls troop deployment
- Manages tower attacks and defense
- Handles damage calculations
- Implements game rules and win conditions

c. Visualization System

- BattlefieldMap for visual representation
- Real-time game state updates

- Combat animations and effects
- Player statistics display

III. Game State Management

- Player session tracking
- Game session management
- Turn-based and real-time modes
- Experience and leveling system

```

1241
1242 // AddPlayerEXP adds EXP to a player and handles level up if applicable
1243 func (m *Manager) AddPlayerEXP(player *models.Player, expAmount int) {
1244     if player == nil {
1245         log.Printf("Warning: Attempted to add EXP to nil player")
1246         return
1247     }
1248
1249     // Add the EXP directly - all bonuses should already be included in expAmount
1250     player.EXP += expAmount
1251
1252     // Check for Level up
1253     leveledUp, newLevel, levelName := configs.CheckLevelUp(player)
1254     if leveledUp {
1255         // If player levels up, notify them and apply new level bonuses
1256         client, exists := m.ClientConnections[player.Username]
1257         if exists {
1258             client.SendMessage(fmt.Sprintf("LEVEL UP! You are now Level %d (%s)!", newLevel, levelName))
1259
1260             // Store HP percentages before applying level bonus
1261             towerHPPPercentages := make([]float64, len(player.Towers))
1262             troopHPPPercentages := make([]float64, len(player.Troops))
1263
1264             for i, tower := range player.Towers {
1265                 if tower.MaxHP > 0 {
1266                     towerHPPPercentages[i] = float64(tower.HP) / float64(tower.MaxHP)
1267                 } else {
1268                     towerHPPPercentages[i] = 0
1269                 }
1270             }
1271
1272             for i, troop := range player.Troops {
1273                 if troop.MaxHP > 0 {
1274                     troopHPPPercentages[i] = float64(troop.HP) / float64(troop.MaxHP)
1275                 } else {
1276                     troopHPPPercentages[i] = 0
1277                 }
1278             }
1279

```

Figure 7 – Player Management

```

1099 // endGame ends the game with a winner
1100 func (m *Manager) endGame(gameID string, winnerIndex int) {
1101     game, exists := m.Games[gameID]
1102     if !exists {
1103         return
1104     }
1105
1106     winner := game.Players[winnerIndex]
1107     loser := game.Players[(winnerIndex+1)%2]
1108
1109     // Base victory EXP
1110     baseExp := 30
1111
1112     // Display detailed EXP summary for winner and get total EXP to award
1113     totalWinnerExp := m.displayExpSummary(gameID, winner, loser, baseExp, true)
1114
1115     // Display detailed EXP summary for loser (they get 0 base EXP for losing)
1116     // But they might get some EXP from destroying towers
1117     totalLoserExp := m.displayExpSummary(gameID, loser, winner, 0, false)
1118
1119     // Award EXP with all bonuses included
1120     m.AddPlayerEXP(winner, totalWinnerExp)
1121     if totalLoserExp > 0 {
1122         m.AddPlayerEXP(loser, totalLoserExp)
1123     }
1124
1125     // Notify players of game outcome
1126     m.notifyPlayers(gameID, fmt.Sprintf("Game over! %s wins!", winner.Username))
1127     m.notifyPlayers(gameID, fmt.Sprintf("%s gained EXP (see summary above)", winner.Username))
1128     if totalLoserExp > 0 {
1129         m.notifyPlayers(gameID, fmt.Sprintf("%s also gained some EXP for towers destroyed", loser.Username))
1130     }
1131
1132     // Save player data
1133     if err := configs.SavePlayerData(winner); err != nil {
1134         log.Printf("Error saving winner data: %v", err)
1135     }

```

Figure 8 – Game End Handling

```

1331 // EndGameTimeExpired handles the game's end when the timer expires
1332 func (m *Manager) EndGameTimeExpired(gameID string) {
1333     game, exists := m.Games[gameID]
1334     if !exists {
1335         return
1336     }
1337
1338     player1 := game.Players[0]
1339     player2 := game.Players[1]
1340
1341     // Count destroyed towers
1342     p1DestroyedTowers := 0
1343     p2DestroyedTowers := 0
1344
1345     for _, tower := range player1.Towers {
1346         if tower.HP <= 0 {
1347             p2DestroyedTowers++
1348         }
1349     }
1350
1351     for _, tower := range player2.Towers {
1352         if tower.HP <= 0 {
1353             p1DestroyedTowers++
1354         }
1355     }
1356
1357     // Determine winner
1358     if p1DestroyedTowers > p2DestroyedTowers {
1359         // Player 1 wins
1360         m.AddPlayerEXP(player1, 30)
1361         m.notifyPlayers(gameID, fmt.Sprintf("Time's up! %s wins by destroying more towers (%d vs %d).",
1362             player1.Username, p1DestroyedTowers, p2DestroyedTowers))
1363         m.notifyPlayers(gameID, fmt.Sprintf("%s gained 30 EXP", player1.Username))
1364     } else if p2DestroyedTowers > p1DestroyedTowers {
1365         // Player 2 wins
1366         m.AddPlayerEXP(player2, 30)
1367         m.notifyPlayers(gameID, fmt.Sprintf("Time's up! %s wins by destroying more towers (%d vs %d).",
1368             player2.Username, p2DestroyedTowers, p1DestroyedTowers))
1369         m.notifyPlayers(gameID, fmt.Sprintf("%s gained 30 EXP", player2.Username))

```

Figure 9 – Time Management

CHAPTER 3

GAME DESIGN

I. Game Modes

```
// Notify players
m.notifyPlayers(gameID, fmt.Sprintf("Game created: %s vs %s (Mode: %s)", player1, player2, gameMode))
m.notifyPlayers(gameID, fmt.Sprintf("Game started! %s's turn", player1))

// Show instructions for simple mode
if gameMode == "simple" {
    m.notifyPlayers(gameID, "\n===== BATTLE GUIDE =====")
    m.notifyPlayers(gameID, "• Players start with 3 units & 3 defensive structures (1 Main, 2 Secondary)")
    m.notifyPlayers(gameID, "• Secondary Defense 1 must fall before targeting Secondary Defense 2 or Main Defense")
    m.notifyPlayers(gameID, "• Units that destroy a structure may immediately launch another attack")
    m.notifyPlayers(gameID, "• Command: DEPLOY <troops_number> <tower_number> to begin assault")
    m.notifyPlayers(gameID, "• Command: STATS to view current units and defenses")
    m.notifyPlayers(gameID, "===== \n")

    // Highlight that it's player 1's turn
    m.notifyPlayers(gameID, "\n-----")
    m.notifyPlayers(gameID, fmt.Sprintf("It's now %s's turn", player1))
    m.notifyPlayers(gameID, "----- \n")
}

// Display the initial stats to show level bonuses are applied
m.displayContinuousStats(gameID)
```

Figure 10 - Simple mode

```
// PlayEnhancedMode starts and manages a game in enhanced mode
func (m *Manager) PlayEnhancedMode(gameID string) error {
    game, exists := m.Games[gameID]
    if !exists {
        return fmt.Errorf("game with ID %s not found", gameID)
    }

    if game.GameMode != "enhanced" {
        return fmt.Errorf("PlayEnhancedMode is only for enhanced mode")
    }

    // Show enhanced mode game rules and instructions
    m.notifyPlayers(gameID, "\n----- ENHANCED MODE RULES -----")
    m.notifyPlayers(gameID, "- No turns. The game lasts 3 minutes, players attack continuously")
    m.notifyPlayers(gameID, "- MANA starts at 5, regenerates 1/sec, max is 10")
    m.notifyPlayers(gameID, "- Use MANA to deploy troops during the game")
    m.notifyPlayers(gameID, "- Critical hits: King Tower 10%, Guard Tower 5% chance")
    m.notifyPlayers(gameID, "- Critical hits deal 20% more damage")
    m.notifyPlayers(gameID, "- You must destroy Guard Tower 1 before attacking Guard Tower 2 or King Tower")
    m.notifyPlayers(gameID, "- Win by destroying King Tower or most towers when time ends")
    m.notifyPlayers(gameID, "- Use DEPLOY <troop_index> <tower_position> to attack")
    m.notifyPlayers(gameID, "----- \n")

    // Display stats for players to see available troops
    m.displayContinuousStats(gameID)

    // Start the game timer in a separate goroutine
    go m.runEnhancedGameLoop(gameID)

    return nil
}
```

Figure 11 – Enhanced mode

II. Game Elements

1. Towers

```
Tower Status:
  nghi: OK [K] King Tower(600) OK [G] Guard Tower(300) OK [G] Guard Tower(300)
  kaze: OK [K] King Tower(600) OK [G] Guard Tower(300) OK [G] Guard Tower(300)
=====
```

Figure 12 – Tower elements

2. Troops

```
Your Troops:
0. ♠ Bishop (Level 1): HP 100/100, ATK 200, DEF 150, MANA 4 - Status: ALIVE
1. ♜ Rook (Level 1): HP 250/250, ATK 200, DEF 200, MANA 5 - Status: ALIVE
2. ♞ Knight (Level 1): HP 200/200, ATK 300, DEF 150, MANA 5 - Status: ALIVE
```

```
Opponent's Troops:
0. ♜ Rook (Level 1): HP 250/250 - Status: ALIVE
1. ♚ Prince (Level 1): HP 500/500 - Status: ALIVE
2. ♛ Queen (Level 1): HP 200/200 - Status: ALIVE
   Special: [Hidden]
```

Figure 13 – Troop elements

III. Combat System

1. Turn Sequence

```
nghi deploys ♜ Rook to attack kaze's ♟ Guard Tower
>>> nghi deploys [T] Rook
>>> [T] Rook advances across the battlefield...
>>> [T] Rook charges toward kaze's Guard Tower!
>>> [T] Rook reaches the target!
>>> ATTACK! [T] Rook attacks kaze's Guard Tower!
nghi's ♜ Rook dealt 150 damage to kaze's ♟ Guard Tower (ATK 200 - DEF 50)
kaze's ♟ Guard Tower has 150/300 HP remaining
```

Figure 14 – Turn Sequence

2. Battlefield Visualization

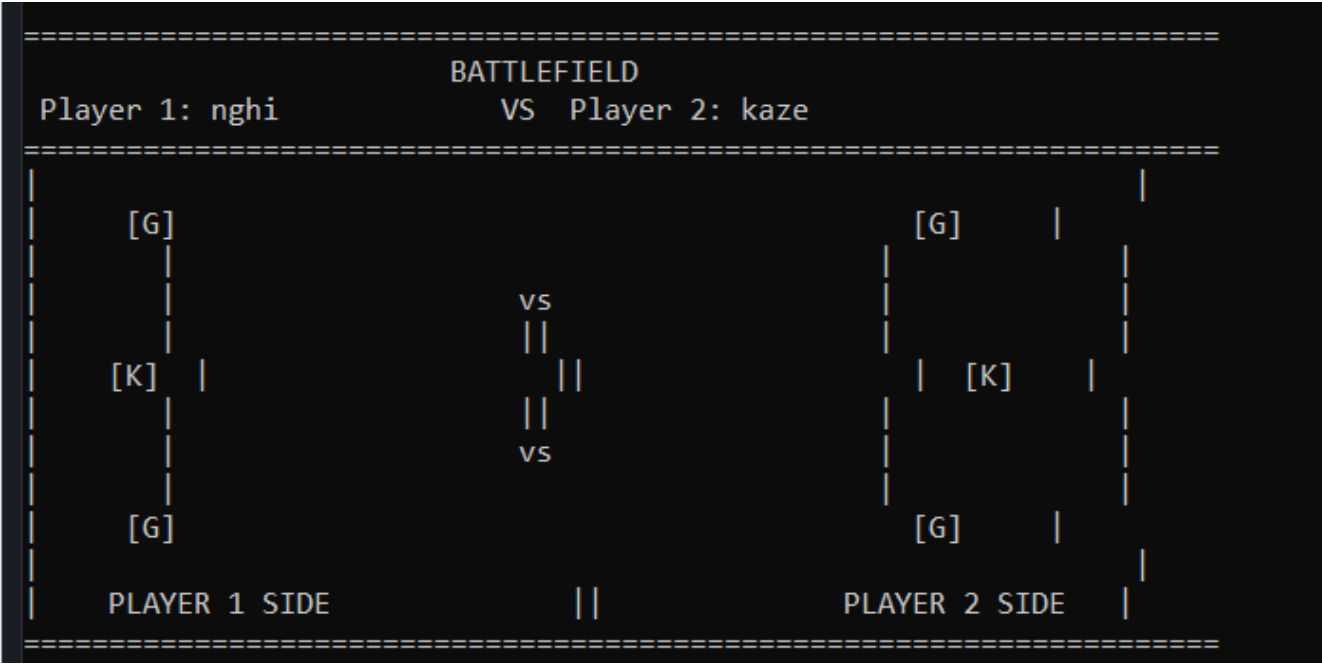


Figure 15 – Battlefield Visualization

CHAPTER IV

CONCLUSION

I. Discussion

The **Text-Based Clash Royale (TCR)** project successfully demonstrates how a multiplayer strategy game can be implemented using Go with a client-server architecture and a terminal-based interface. Despite the limitations of a text-only environment, the game manages to replicate the core mechanics of tower defense, troop deployment, and real-time or turn-based combat.

Key accomplishments include:

- Implementing a stable **TCP-based communication layer** between multiple clients and the server.
- Designing modular components such as the **Game Logic**, **Battle System**, **Player Management**, and a **Configuration System** to allow flexibility and future scalability.
- Supporting two distinct game modes—**Simple (Turn-based)** and **Enhanced (Real-time)**—with dynamic rules, resource systems, and combat strategies.
- Creating a **text-based UI** that clearly conveys game state, available actions, and battle outcomes.

While TCR operates successfully in its current form, several challenges were encountered and overcome:

- **Real-time interaction over TCP** introduced complexity in synchronizing player actions without race conditions or deadlocks. This was addressed using Go's built-in concurrency primitives.
- **Game balance and fairness** needed careful tuning, especially when introducing new mechanics like mana and critical hits. External configuration files helped streamline testing different balance setups.

- **Player experience in a text-based environment** required thoughtful design of feedback messages and input prompts to ensure the game remained engaging and comprehensible.

Overall, the project demonstrates the capability of Go in handling concurrent, networked applications and shows how game logic can be structured cleanly even without a graphical interface.

II. Future Improvements

While the project achieved its core goals, there are several areas where it could be enhanced in future iterations. One major direction is improving the user interface. Although the current text-based display works, adopting a terminal user interface (TUI) library such as `tview` could greatly enhance the visual feedback and interactivity, allowing players to see troop layouts, health bars, and battle logs more clearly. In terms of networking, adding support for reconnections and more robust handling of disconnections would make the game more resilient, especially during real-time matches.

Feature-wise, the game could benefit from the introduction of more advanced mechanics such as deck-building, troop upgrades, or AI-controlled opponents for single-player mode. These additions would not only expand gameplay depth but also make the game accessible when no other players are available. Enhancing scalability is also important—implementing a matchmaking system based on player performance or ranking would allow fairer and faster match pairing, and optimizing concurrency handling could improve support for many simultaneous matches.

Furthermore, player progression could be expanded by incorporating persistent profiles that track experience, wins, and match history. This would make the game more rewarding over time and enable the possibility of features like leaderboards or seasonal challenges. Finally, transitioning from a pure terminal-based system to a browser-accessible version (via websockets or a lightweight web interface) could open the game to a wider audience and set the stage for further development.

In conclusion, TCR provides a strong technical foundation for a multiplayer strategy game and presents numerous opportunities for future growth. With thoughtful extensions in UI, gameplay

mechanics, scalability, and persistence, the project can evolve into a rich and more complete gaming experience—all while remaining grounded in Go’s strengths in networking and concurrency.

III. References

1. Go Documentation. (n.d.). The Go Programming Language. <https://go.dev/doc/>
2. TCP/IP Protocol Suite. (n.d.). Transmission Control Protocol/Internet Protocol. <https://www.ietf.org/rfc/rfc793.txt>
3. Clash Royale. (2016). Supercell. <https://clashroyale.com/>
4. Go Concurrency Patterns. (n.d.). Go by Example. <https://gobyexample.com/goroutines>
5. JSON Specification. (2017). The JavaScript Object Notation Data Interchange Format. <https://www.rfc-editor.org/rfc/rfc8259>
6. Go Standard Library. (n.d.). Package sync. <https://pkg.go.dev/sync>
7. Go Network Programming. (n.d.). Package net. <https://pkg.go.dev/net>
8. Go Error Handling. (n.d.). Error handling and Go. <https://go.dev/blog/error-handling-and-go>
9. Game Development Patterns. (n.d.). Game Programming Patterns. <http://gameprogrammingpatterns.com/>
10. Go Testing. (n.d.). Testing in Go. <https://go.dev/doc/tutorial/add-a-test>
11. Go Memory Management. (n.d.). Go Memory Management. <https://go.dev/doc/gc-guide>
12. Go Security Best Practices. (n.d.). Go Security. <https://go.dev/doc/security>
13. Go Performance Optimization. (n.d.). Go Performance. https://go.dev/doc/effective_go#performance
14. Go Command Line Tools. (n.d.). Go Command. <https://pkg.go.dev/cmd/go>
15. Go Module System. (n.d.). Go Modules. <https://go.dev/doc/modules/gomod-ref>